

```
In [1]: import pandas as pd

# Load the dataset
df = pd.read_csv("file:///Users/maoqingwang/Downloads/first_25000_rows.csv")

# Display basic info and first few rows to understand the structure
df.info(), df.head()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 74 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ts_recv                5000 non-null   object
1   ts_event                5000 non-null   object
2   rtype                  5000 non-null   int64
3   publisher_id            5000 non-null   int64
4   instrument_id           5000 non-null   int64
5   action                 5000 non-null   object
6   side                   5000 non-null   object
7   depth                  5000 non-null   int64
8   price                  5000 non-null   float64
9   size                   5000 non-null   int64
10  flags                   5000 non-null   int64
11  ts_in_delta             5000 non-null   int64
12  sequence                5000 non-null   int64
13  bid_px_00               5000 non-null   float64
14  ask_px_00               5000 non-null   float64
15  bid_sz_00               5000 non-null   int64
16  ask_sz_00               5000 non-null   int64
17  bid_ct_00               5000 non-null   int64
18  ask_ct_00               5000 non-null   int64
19  bid_px_01               5000 non-null   float64
20  ask_px_01               5000 non-null   float64
21  bid_sz_01               5000 non-null   int64
22  ask_sz_01               5000 non-null   int64
23  bid_ct_01               5000 non-null   int64
24  ask_ct_01               5000 non-null   int64
25  bid_px_02               5000 non-null   float64
26  ask_px_02               5000 non-null   float64
27  bid_sz_02               5000 non-null   int64
28  ask_sz_02               5000 non-null   int64
29  bid_ct_02               5000 non-null   int64
30  ask_ct_02               5000 non-null   int64
31  bid_px_03               5000 non-null   float64
32  ask_px_03               5000 non-null   float64
33  bid_sz_03               5000 non-null   int64
34  ask_sz_03               5000 non-null   int64
35  bid_ct_03               5000 non-null   int64
36  ask_ct_03               5000 non-null   int64
37  bid_px_04               5000 non-null   float64
38  ask_px_04               5000 non-null   float64
39  bid_sz_04               5000 non-null   int64

```

```
40 ask_sz_04      5000 non-null int64
41 bid_ct_04      5000 non-null int64
42 ask_ct_04      5000 non-null int64
43 bid_px_05      5000 non-null float64
44 ask_px_05      5000 non-null float64
45 bid_sz_05      5000 non-null int64
46 ask_sz_05      5000 non-null int64
47 bid_ct_05      5000 non-null int64
48 ask_ct_05      5000 non-null int64
49 bid_px_06      5000 non-null float64
50 ask_px_06      5000 non-null float64
51 bid_sz_06      5000 non-null int64
52 ask_sz_06      5000 non-null int64
53 bid_ct_06      5000 non-null int64
54 ask_ct_06      5000 non-null int64
55 bid_px_07      5000 non-null float64
56 ask_px_07      5000 non-null float64
57 bid_sz_07      5000 non-null int64
58 ask_sz_07      5000 non-null int64
59 bid_ct_07      5000 non-null int64
60 ask_ct_07      5000 non-null int64
61 bid_px_08      5000 non-null float64
62 ask_px_08      5000 non-null float64
63 bid_sz_08      5000 non-null int64
64 ask_sz_08      5000 non-null int64
65 bid_ct_08      5000 non-null int64
66 ask_ct_08      5000 non-null int64
67 bid_px_09      5000 non-null float64
68 ask_px_09      5000 non-null float64
69 bid_sz_09      5000 non-null int64
70 ask_sz_09      5000 non-null int64
71 bid_ct_09      5000 non-null int64
72 ask_ct_09      5000 non-null int64
73 symbol         5000 non-null object
dtypes: float64(21), int64(48), object(5)
memory usage: 2.8+ MB
```

Out[1]: (None,

		ts_recv	ts_event	rtype	\
0	2024-10-21T11:54:29.221230963Z	2024-10-21T11:54:29.221064336Z		10	
1	2024-10-21T11:54:29.223936626Z	2024-10-21T11:54:29.223769812Z		10	
2	2024-10-21T11:54:29.225196809Z	2024-10-21T11:54:29.225030400Z		10	
3	2024-10-21T11:54:29.712600612Z	2024-10-21T11:54:29.712434212Z		10	
4	2024-10-21T11:54:29.764839221Z	2024-10-21T11:54:29.764673165Z		10	

	publisher_id	instrument_id	action	side	depth	price	size	...	\
0	2	38	C	B	1	233.62	2	...	
1	2	38	A	B	0	233.67	2	...	
2	2	38	A	B	0	233.67	3	...	
3	2	38	A	B	2	233.52	200	...	
4	2	38	C	B	2	233.52	200	...	

	ask_sz_08	bid_ct_08	ask_ct_08	bid_px_09	ask_px_09	bid_sz_09	\
0	155	1	7	233.25	234.13	55	
1	155	1	7	233.25	234.13	55	
2	155	1	7	233.25	234.13	55	
3	155	1	7	233.25	234.13	55	
4	155	1	7	233.25	234.13	55	

	ask_sz_09	bid_ct_09	ask_ct_09	symbol
0	400	2	1	AAPL
1	400	2	1	AAPL
2	400	2	1	AAPL
3	400	2	1	AAPL
4	400	2	1	AAPL

[5 rows x 74 columns])

In [2]: df

Out [2]:

	ts_recv	ts_event	rtype	publisher_id	instrument_id	action	side	depth	price	size	...	as
0	2024-10-21T11:54:29.221230963Z	2024-10-21T11:54:29.221064336Z	10	2	38	C	B	1	233.62	2	...	
1	2024-10-21T11:54:29.223936626Z	2024-10-21T11:54:29.223769812Z	10	2	38	A	B	0	233.67	2	...	
2	2024-10-21T11:54:29.225196809Z	2024-10-21T11:54:29.225030400Z	10	2	38	A	B	0	233.67	3	...	
3	2024-10-21T11:54:29.712600612Z	2024-10-21T11:54:29.712434212Z	10	2	38	A	B	2	233.52	200	...	
4	2024-10-21T11:54:29.764839221Z	2024-10-21T11:54:29.764673165Z	10	2	38	C	B	2	233.52	200	...	
...	...	...	...	...	...	...	...	...	...	...	...	
4995	2024-10-21T13:04:16.583694069Z	2024-10-21T13:04:16.583527688Z	10	2	38	A	B	2	233.46	200	...	
4996	2024-10-21T13:04:17.976627074Z	2024-10-21T13:04:17.976461017Z	10	2	38	A	A	1	233.69	200	...	
4997	2024-10-21T13:04:20.085804687Z	2024-10-21T13:04:20.085638629Z	10	2	38	C	B	2	233.46	200	...	
4998	2024-10-21T13:04:20.085817362Z	2024-10-21T13:04:20.085651109Z	10	2	38	A	B	3	233.44	200	...	
4999	2024-10-21T13:04:20.131008559Z	2024-10-21T13:04:20.130842270Z	10	2	38	A	A	0	233.56	14	...	

5000 rows × 74 columns

```

In [7]: # Re-clean the data using user-provided view as confirmation
# Convert timestamps again in case the prior state was reset
df['ts_event'] = pd.to_datetime(df['ts_event'], utc=True)
df.sort_values(by='ts_event', inplace=True)
df.reset_index(drop=True, inplace=True)

# Drop duplicates just in case
df.drop_duplicates(inplace=True)
df

```

Out [7]:

	ts_recv	ts_event	rtype	publisher_id	instrument_id	action	side	depth	price	size	...
0	2024-10-21T11:54:29.221230963Z	2024-10-21 11:54:29.221064336+00:00	10	2	38	C	B	1	233.62	2	...
1	2024-10-21T11:54:29.223936626Z	2024-10-21 11:54:29.223769812+00:00	10	2	38	A	B	0	233.67	2	...
2	2024-10-21T11:54:29.225196809Z	2024-10-21 11:54:29.225030400+00:00	10	2	38	A	B	0	233.67	3	...
3	2024-10-21T11:54:29.712600612Z	2024-10-21 11:54:29.712434212+00:00	10	2	38	A	B	2	233.52	200	...
4	2024-10-21T11:54:29.764839221Z	2024-10-21 11:54:29.764673165+00:00	10	2	38	C	B	2	233.52	200	...
...	...	...	...	...	...	...	...	...	...	...	...
4995	2024-10-21T13:04:16.583694069Z	2024-10-21 13:04:16.583527688+00:00	10	2	38	A	B	2	233.46	200	...
4996	2024-10-21T13:04:17.976627074Z	2024-10-21 13:04:17.976461017+00:00	10	2	38	A	A	1	233.69	200	...
4997	2024-10-21T13:04:20.085804687Z	2024-10-21 13:04:20.085638629+00:00	10	2	38	C	B	2	233.46	200	...
4998	2024-10-21T13:04:20.085817362Z	2024-10-21 13:04:20.085651109+00:00	10	2	38	A	B	3	233.44	200	...
4999	2024-10-21T13:04:20.131008559Z	2024-10-21 13:04:20.130842270+00:00	10	2	38	A	A	0	233.56	14	...

5000 rows × 74 columns

```

In [19]: # ofi_feature_engineering.py

import pandas as pd
import numpy as np
from sklearn.decomposition import PCA

def load_and_clean_data(path):
    df = pd.read_csv(path)
    df['ts_event'] = pd.to_datetime(df['ts_event'], utc=True)
    df.sort_values('ts_event', inplace=True)
    df.drop_duplicates(inplace=True)
    df.reset_index(drop=True, inplace=True)

```

```

return df

def compute_best_level_ofi(df):
    df['OFI_best_level'] = df['bid_sz_00'].diff().fillna(0) - df['ask_sz_00'].diff().fillna(0)
    return df

def compute_multi_level_ofi(df, depth=10):
    delta_bid = [df[f'bid_sz_{i}'].diff().fillna(0) for i in range(depth)]
    delta_ask = [df[f'ask_sz_{i}'].diff().fillna(0) for i in range(depth)]
    df['OFI_multi_level'] = sum(delta_bid) - sum(delta_ask)
    return df

def compute_integrated_ofi(df, depth=10):
    features = [df[f'bid_sz_{i}'].diff().fillna(0) - df[f'ask_sz_{i}'].diff().fillna(0) for i in range(depth)]
    ofi_matrix = pd.concat(features, axis=1)
    ofi_matrix.columns = [f'OFI_L_{i}' for i in range(depth)]
    pca = PCA(n_components=1)
    df['OFI_integrated'] = pca.fit_transform(ofi_matrix)
    return df

def add_lagged_features(df):
    df['OFI_best_level_lag1'] = df['OFI_best_level'].shift(1)
    df['OFI_multi_level_lag1'] = df['OFI_multi_level'].shift(1)
    df['OFI_integrated_lag1'] = df['OFI_integrated'].shift(1)
    return df

```

```

In [10]: #Compute Best Level OFI, immediately accessible liquidity shift
df['delta_bid_sz_00'] = df['bid_sz_00'].diff().fillna(0)
df['delta_ask_sz_00'] = df['ask_sz_00'].diff().fillna(0)
df['OFI_best_level'] = df['delta_bid_sz_00'] - df['delta_ask_sz_00']

# Display the DataFrame using standard output method due to missing ace_tools module
ofi_best_cleaned_df = df[['ts_event', 'symbol', 'bid_sz_00', 'ask_sz_00', 'delta_bid_sz_00', 'delta_ask_sz_00', 'OFI_best_level']]

# Show the first few rows directly
ofi_best_cleaned_df.head()

```

Out[10]:

	ts_event	symbol	bid_sz_00	ask_sz_00	delta_bid_sz_00	delta_ask_sz_00	OFI_best_level
0	2024-10-21 11:54:29.221064336+00:00	AAPL	139	200	0.0	0.0	0.0
1	2024-10-21 11:54:29.223769812+00:00	AAPL	141	200	2.0	0.0	2.0
2	2024-10-21 11:54:29.225030400+00:00	AAPL	144	200	3.0	0.0	3.0
3	2024-10-21 11:54:29.712434212+00:00	AAPL	144	200	0.0	0.0	0.0
4	2024-10-21 11:54:29.764673165+00:00	AAPL	144	200	0.0	0.0	0.0

In [11]:

```
#multi level OFI

# Step 1: Compute multi-level bid and ask size differences (for levels 0 to 9)
levels = range(10)
delta_bid = [df[f'bid_sz_{i}'].diff().fillna(0) for i in levels]
delta_ask = [df[f'ask_sz_{i}'].diff().fillna(0) for i in levels]

# Step 2: Compute multi-level OFI by summing differences across all 10 levels
df['OFI_multi_level'] = sum(delta_bid) - sum(delta_ask)

# Step 3: Extract relevant columns
multi_level_ofi_df = df[['ts_event', 'symbol', 'OFI_multi_level']].copy()
multi_level_ofi_df.head()
```

Out[11]:

	ts_event	symbol	OFI_multi_level
0	2024-10-21 11:54:29.221064336+00:00	AAPL	0.0
1	2024-10-21 11:54:29.223769812+00:00	AAPL	2.0
2	2024-10-21 11:54:29.225030400+00:00	AAPL	3.0
3	2024-10-21 11:54:29.712434212+00:00	AAPL	200.0
4	2024-10-21 11:54:29.764673165+00:00	AAPL	-200.0

In [12]:

```
#Integrated OFI
from sklearn.decomposition import PCA

# Step 1: Prepare multi-level OFI components for PCA (bid_sz_00 to 09, ask_sz_00 to 09 diffs)
multi_level_features = []

for i in range(10):
    bid_diff = df[f'bid_sz_{i}'].diff().fillna(0)
```

```
ask_diff = df[f'ask_sz_{i}'].diff().fillna(0)
multi_level_features.append(bid_diff - ask_diff)

# Combine into feature matrix
ofi_matrix = pd.concat(multi_level_features, axis=1)
ofi_matrix.columns = [f'OFI_L{i}' for i in range(10)]

# Step 2: Apply PCA
pca = PCA(n_components=1)
df['OFI_integrated'] = pca.fit_transform(ofi_matrix)

# Step 3: Display
integrated_ofi_df = df[['ts_event', 'symbol', 'OFI_integrated']].copy()
integrated_ofi_df.head()
```

Out[12]:

	ts_event	symbol	OFI_integrated
0	2024-10-21 11:54:29.221064336+00:00	AAPL	0.065551
1	2024-10-21 11:54:29.223769812+00:00	AAPL	0.081956
2	2024-10-21 11:54:29.225030400+00:00	AAPL	0.090158
3	2024-10-21 11:54:29.712434212+00:00	AAPL	100.073298
4	2024-10-21 11:54:29.764673165+00:00	AAPL	-99.942195

In [13]:

```
#Cross Asset OFI, can't achive bc just one asset, but could use Lagged OFI
# Step 1: Create lagged versions of best-level, multi-level, and integrated OFIs
df['OFI_best_level_lag1'] = df['OFI_best_level'].shift(1)
df['OFI_multi_level_lag1'] = df['OFI_multi_level'].shift(1)
df['OFI_integrated_lag1'] = df['OFI_integrated'].shift(1)

# Step 2: Extract relevant columns for display
lagged_ofi_df = df[['ts_event', 'symbol',
                    'OFI_best_level', 'OFI_best_level_lag1',
                    'OFI_multi_level', 'OFI_multi_level_lag1',
                    'OFI_integrated', 'OFI_integrated_lag1']]

lagged_ofi_df.head()
```

Out[13]:

	ts_event	symbol	OFI_best_level	OFI_best_level_lag1	OFI_multi_level	OFI_multi_level_lag1	OFI_integrated	OFI
0	2024-10-21 11:54:29.221064336+00:00	AAPL	0.0	NaN	0.0	NaN	0.065551	
1	2024-10-21 11:54:29.223769812+00:00	AAPL	2.0	0.0	2.0	0.0	0.081956	
2	2024-10-21 11:54:29.225030400+00:00	AAPL	3.0	2.0	3.0	2.0	0.090158	
3	2024-10-21 11:54:29.712434212+00:00	AAPL	0.0	3.0	200.0	3.0	100.073298	
4	2024-10-21 11:54:29.764673165+00:00	AAPL	0.0	0.0	-200.0	200.0	-99.942195	

In [15]:

```

from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Prepare the predictor matrix X and response y
X = df[['OFI_best_level_lag1', 'OFI_multi_level_lag1', 'OFI_integrated_lag1']].fillna(0)
y = df['return'].fillna(0)

# Use a pipeline to scale the data and apply LassoCV with cross-validation
lasso_model = make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=42))
lasso_model.fit(X, y)

# Extract coefficients
lasso_coefficients = lasso_model.named_steps['lassocv'].coef_
lasso_intercept = lasso_model.named_steps['lassocv'].intercept_

lasso_coefficients, lasso_intercept

```

Out[15]: (array([0., 0., 0.]), -1.4553549139755263e-07)

In [16]:

```

from sklearn.linear_model import Lasso
import numpy as np

# Manually set a smaller alpha (regularization strength) to allow some nonzero coefficients
manual_lasso = make_pipeline(StandardScaler(), Lasso(alpha=1e-6))
manual_lasso.fit(X, y)

# Extract manually tuned coefficients

```

```
manual_lasso_coefficients = manual_lasso.named_steps['lasso'].coef_  
manual_lasso_intercept = manual_lasso.named_steps['lasso'].intercept_  
  
manual_lasso_coefficients, manual_lasso_intercept
```

Out[16]: (array([7.57900321e-07, 0.00000000e+00, 0.00000000e+00]),
-1.4553549139755263e-07)

```
In [14]: # Re-import numpy since it was missing  
import numpy as np  
import statsmodels.api as sm  
  
# Re-run the midprice and return calculations  
df['midprice'] = (df['bid_px_00'] + df['ask_px_00']) / 2  
df['log_midprice'] = np.log(df['midprice'])  
df['return'] = df['log_midprice'].diff().fillna(0)  
  
# Prepare regression variables  
X = df[['OFI_best_level_lag1', 'OFI_multi_level_lag1', 'OFI_integrated_lag1']].fillna(0)  
X = sm.add_constant(X)  
y = df['return']  
  
# Fit OLS model  
model = sm.OLS(y, X).fit()  
model.summary()
```

Out [14]:

OLS Regression Results

Dep. Variable:	return	R-squared:	0.001
Model:	OLS	Adj. R-squared:	0.001
Method:	Least Squares	F-statistic:	2.200
Date:	Fri, 02 May 2025	Prob (F-statistic):	0.0859
Time:	16:46:32	Log-Likelihood:	42166.
No. Observations:	5000	AIC:	-8.432e+04
Df Residuals:	4996	BIC:	-8.430e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-1.456e-07	7.45e-07	-0.196	0.845	-1.61e-06	1.31e-06
OFI_best_level_lag1	9.334e-09	4.02e-09	2.323	0.020	1.46e-09	1.72e-08
OFI_multi_level_lag1	2.977e-11	2.81e-09	0.011	0.992	-5.48e-09	5.54e-09
OFI_integrated_lag1	1.198e-09	1.18e-09	1.012	0.311	-1.12e-09	3.52e-09

Omnibus:	1532.936	Durbin-Watson:	2.370
Prob(Omnibus):	0.000	Jarque-Bera (JB):	277564.992
Skew:	-0.069	Prob(JB):	0.00
Kurtosis:	39.501	Cond. No.	631.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []: